

1). Multiple Linear Regression:

LinearRegression

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_Train, Y_Train)

weight = regressor.coef_
bias = regressor.intercept_
Y_Predict = regressor.predict(X_Test)
```

Parameters (set when creating the model)

```
model = LinearRegression(
    fit_intercept=True,
    copy_X=True,
    n_jobs=None,
    positive=False,
    tol=1e-6
)
```

Attributes (available after fitting the model)

```
model.coef_      # Coefficients (weights)
model.intercept_ # Intercept (bias term)
model.rank_      # Rank of matrix X
model.singular_  # Singular values of X
model.n_features_in_ # Number of features seen during fit
model.feature_names_in_ # Feature names (if dataset had them)
```

```
from sklearn.metrics import r2_score
R_Score = r2_score(Y_Test, Y_Predict)
R_Score
```

Multiple Linear Regression	
r2_score	0.9358680970046243

2). Support Vector Machine

Support Vector Regression applies Support Vector Machine principles to regression problems. Model performance can vary significantly depending on kernel type, C value, epsilon, and feature scaling

```
from sklearn.svm import SVR
regressor = SVR(kernel = 'linear', C=0.1)
regressor.fit(X_Train, Y_Train)

bias = regressor.intercept_
regressor.n_support_
regressor.support_
Y_Pred = regressor.predict(X_Test)
```

Different SVR models

- *svr_lowC* = SVR(C=0.1, epsilon=0.1, kernel='rbf')
- *svr_highC* = SVR(C=100, epsilon=0.1, kernel='rbf')
- *svr_highEps* = SVR(C=1, epsilon=1.0, kernel='rbf')

Parameters (set when creating the model)

```
SVR(
    kernel='rbf',
    degree=3,
    gamma='scale',
    coef0=0.0,
    tol=1e-3,
    C=1.0,
    epsilon=0.1,
    shrinking=True,
    cache_size=200,
    verbose=False,
    max_iter=-1
)
```

SVR Kernel options

- *linear* → Linear kernel
- *poly* → Polynomial kernel
- *rbf* → Radial Basis Function (default)
- *sigmoid* → Sigmoid kernel
- *precomputed* → Use a custom kernel matrix

👉 Most common: *rbf* for non-linear data, *linear* for simple linear trends.

Attributes (available after fitting the model)

```
svr.support_      # Indices of support vectors
svr.support_vectors_ # Support vectors themselves
svr.n_support_    # Number of support vectors per class
svr.dual_coef_    # Coefficients of support vectors in decision function
svr.intercept_    # Intercept term(s)
```

```
from sklearn.metrics import r2_score
R_Score = r2_score(Y_Test, Y_Pred)
R_Score
```

Support Vector Machine (With StandardScaler)						
	Hyper Tuning parameters					
			Kernel			
Model Name	<i>C</i>	<i>epsilon</i>	Linear (R ² values)	Poly (R ² values)	Rbf (R ² values)	Sigmoid (R ² values)
<i>svr_lowC</i>	0.1	0.1	-0.0573	-0.0574	-0.0574	-0.0574
<i>svr_highC</i>	100	0.1	0.1064	-0.0198	-0.0507	-0.0304
<i>svr_highEps</i>	1	1	-0.0556	-0.0198	-0.0574	-0.0572

Support Vector Machine (Without StandardScaler)						
	Hyper Tuning parameters					
			Kernel			
Model Name	<i>C</i>	<i>epsilon</i>	Linear (R ² values)	Poly (R ² values)	Rbf (R ² values)	Sigmoid (R ² values)
<i>svr_lowC</i>	0.1	0.1	0.9375	-0.0568	-0.0574	-0.0574
<i>svr_highC</i>	100	0.1	-357.0795	0.4656	-0.0302	-0.0587
<i>svr_highEps</i>	1	1	0.8950	-0.0508	-0.0573	-0.0574

Finding: For the supplied values, a linear SVR without StandardScaler and C=0.1 achieved the highest reported R² score of 0.9375.

3). Decision Tree

```
from sklearn.tree import DecisionTreeRegressor
regressor = DecisionTreeRegressor()
regressor = regressor.fit(X_Train, Y_Train)

import matplotlib.pyplot as plt
from sklearn import tree
tree.plot_tree(regressor)
plt.show()
```

Parameters (set when creating the model)

Parameters and Their Options

- *criterion*
 - *"squared_error"* (default)
 - *"friedman_mse"*
 - *"absolute_error"*
 - *"poisson"*
- *splitter*
 - *"best"* (default)
 - *"random"*
- *max_depth*
 - *None* (default, unlimited depth)
 - *Any positive integer* (limits depth)
- *min_samples_split*
 - *Integer ≥ 2* (default: 2)
 - *Float between 0.0 and 1.0* (fraction of samples)
- *min_samples_leaf*
 - *Integer ≥ 1* (default: 1)
 - *Float between 0.0 and 0.5* (fraction of samples)
- *min_weight_fraction_leaf*
 - *Float between 0.0 and 0.5* (default: 0.0)
- *max_features*
 - *None* (default, all features)
 - *"auto"*
 - *"sqrt"*
 - *"log2"*
 - *Integer or float* (number or fraction of features)
- *random_state*
 - *None* (default)
 - *Integer* (for reproducibility)
- *max_leaf_nodes*
 - *None* (default, unlimited)
 - *Integer ≥ 1* (limits number of leaf nodes)
- *min_impurity_decrease*

- *Float ≥ 0.0 (default: 0.0)*
- *ccp_alpha*
 - *Float ≥ 0.0 (default: 0.0)*
 - *Used for pruning*

Attributes (available after fitting the model)

*feature_importances_
n_features_in_
feature_names_in_
n_outputs_
tree_*

Decision Tree					
Hyper Tuning parameters					
		criterion			
splitter	max_features	squared_error (R ² values)	friedman_mse (R ² values)	absolute_error (R ² values)	poisson (R ² values)
best	None	0.9357	0.9010	0.9330	0.9233
best	sqrt	0.5072	0.6671	0.0804	-0.3046
best	log2	0.4207	0.9298	0.6942	0.0573
random	None	0.5910	0.8898	0.9299	0.9022
random	sqrt	0.6565	-0.4242	0.7167	-0.4898
random	log2	-0.3818	0.6593	-1.2751	0.6409

Finding: Using criterion="squared_error", splitter="best" and max_features=None produced the highest reported R² score of 0.9357.

Comparison table of the three models

Model	Key Parameters / Setup	R ² Score
Multiple Linear Regression	Default setup	0.9358
Support Vector Regression	Linear SVR, C=0.1, without StandardScaler	0.9375
Decision Tree Regressor	criterion="squared_error", splitter="best", max_features=None, C=0.1	0.9357

✅ **Best Model: Support Vector Regression (SVR)** with C=0.1 (no StandardScaler), achieving the highest R² score of 0.9375.